

Introduction

If you've built anything with an LLM-powered agent, you've hit this wall: the agent gives you an answer with total confidence, and the answer is wrong. Not "I'm not sure, but..." wrong — flat-out, stated-as-fact wrong. No error message, no hesitation, nothing to signal that something broke.

This matters more than it sounds like it should. As agents move from answering trivia questions to writing code, calling APIs, and taking actions with real consequences, a silent wrong answer is far more expensive than a slow right one. Interviewers and engineering teams alike are starting to care less about whether you can call an LLM and more about whether you can build a system that notices when it's wrong and does something about it.

That's exactly what the Reflexion pattern solves. It's a simple idea with an outsized effect: let the agent grade its own work, write down what went wrong in plain language, and hand that note to itself on the next attempt. No retraining, no new model weights, just a smarter loop.

In this lesson we'll build the intuition from scratch, walk through a concrete coding bug the way an agent would actually encounter it, and compare Reflexion against the two things people confuse it with — plain retrying and fine-tuning.

Problem Overview

Strip away the AI framing for a second and the problem is this: you have a system that produces an output, and sometimes that output is wrong. You want the system to improve its own output across multiple attempts, using only information it generates itself — no human in the loop correcting it, no additional training data, no gradient updates.

Formally, Reflexion frames this as three cooperating roles:

- **Actor** — the part that actually attempts the task (writes the code, answers the question, takes the action).
- **Evaluator** — the part that checks the actor's output against some notion of correctness (a test suite, a rubric, a set of constraints) and produces a verdict.
- **Self-reflection** — the part that looks at a *failed* verdict and writes a short, specific, natural-language note about what went wrong and why.

That note then gets folded into the next attempt's prompt. The actor doesn't start over — it starts from the lesson. The loop repeats until the evaluator is satisfied or you hit a cap on the number of attempts.

Example

Let's make this concrete with the example from the lesson: an agent is asked to write a Python function that sums a list of numbers.

Attempt 1 — the agent writes:

```
def sum_list(nums):  
    total = nums[0]  
    for n in nums[1:]:  
        total += n  
    return total
```

This works for `[1, 2, 3]` (returns `6`) and even for a single-item list `[5]` (returns `5`). It looks correct — until the evaluator runs it against `[]`, an empty list, and it throws `IndexError: list index out of range`.

Evaluator output: "Failed on input `[]` with `IndexError`."

Reflection note (written by the agent, in plain language): "The function assumes the list has at least one element by reading `nums[0]` directly. It needs to handle the empty-list case before indexing."

Attempt 2 — with that note prepended to the prompt, the agent writes:

```
def sum_list(nums):  
    total = 0  
    for n in nums:  
        total += n  
    return total
```

This version starts from `0` and never indexes directly, so it handles `[]` (returns `0`), `[5]` (returns `5`), and `[1, 2, 3]` (returns `6`) correctly. The evaluator reruns all cases, everything passes, and the loop stops.

Notice what actually changed between attempts: not the agent's general coding ability, not its "training," just the information available to it at generation time.

Intuition

Here's how an experienced engineer would arrive at this pattern, before ever hearing the word "Reflexion."

Think about writing an essay. Nobody nails a first draft. You write it, you read it back, you circle the weak paragraph, and you rewrite *just that part* — you don't burn the whole essay and start from a blank page. The reason a second draft is better isn't that you got smarter between draft one and draft two. It's that you now have specific, targeted information about what was wrong with draft one.

Now split that single person into three roles: a writer who drafts, an editor who circles the weak paragraph, and a tutor who writes "fix this next time" in the margin. That's the entire Reflexion pattern. The actor is the writer, the evaluator is the editor, and the self-reflection step is the tutor's margin note.

The key insight — the thing that separates this from just "try again" — is that the note has to be *specific*. "This was wrong" is useless. "This crashed on an empty list because you indexed `nums[0]` before checking length" is something the next attempt can actually act on. Specificity is what turns a retry into a correction.

And the memory question — where does this note live? — has a clean answer once you separate two timescales:

- **Short-term memory** is the sticky note on your desk. It holds the reflection for *this* task only, and gets thrown away once the task succeeds or you give up.
- **Long-term memory** is the notebook you keep for weeks. A lesson from last Tuesday's task ("don't assume non-empty input") can quietly prevent a mistake in an unrelated task today.

Most simple Reflexion setups only need short-term memory. Long-term memory is where it gets genuinely powerful, because the agent stops re-learning the same lesson on every new task.

Brute Force Solution

The "brute force" version of self-correction is **plain retry**: run the actor again with the same prompt (maybe with a higher temperature) and hope randomness lands differently.

Idea: Sampling is stochastic, so a second sample might avoid the mistake the first one made.

Algorithm: 1. Run the actor. 2. If it fails, run the actor again, unchanged. 3. Repeat up to N times.

Advantages: - Trivial to implement — no evaluator, no reflection step, no extra prompt engineering. - Actually works reasonably well for failures that are genuinely random (an unlucky token sample, a flaky formatting slip).

Disadvantages: - No memory of *why* it failed, so it can make the exact same mistake on every attempt. - Wastes attempts on systematic bugs — a logic error like "forgot to handle the empty list" isn't random, and rerolling the dice won't fix it. - No principled stopping condition beyond a fixed retry count.

Complexity (informal): Time cost scales linearly with number of retries, with no guarantee of improvement per retry — it's $O(N)$ attempts with no monotonic progress. Space cost is minimal since nothing is retained between attempts.

Optimal Solution

Reflexion is what you get when you replace "retry blindly" with "retry informed." The loop looks like this:

Step	Role	Action
1	Actor	Attempts the task, produces an output
2	Evaluator	Scores the output against a correctness check
3	Self-reflection	If the score fails, writes a specific note explaining what broke and why
4	Memory	Stores the note (short-term for this task, optionally long-term across tasks)
5	Actor	Re-attempts, with the note included in its next prompt
—	Loop	Repeat 2–5 until evaluator passes, or attempt limit is hit

The two design decisions that make or break this pattern:

1. **The stopping condition.** Without a hard cap, the loop can burn through your token budget "refining" an answer that was already good enough, chasing marginal improvements forever. Cap it — three to five attempts is a reasonable default — and accept the best attempt after that.
2. **The quality of the reflection.** A vague note ("that was wrong") gives the next attempt nothing to work with, so you're effectively back to plain retry with extra steps. A useful note names the specific failure — which test failed, which line broke, which assumption was violated.

Get both right, and the loop converges fast because every attempt after the first is informed rather than random.

Python Code

```

from dataclasses import dataclass, field
from typing import Callable, Optional

@dataclass
class ReflexionResult:
    success: bool
    output: Optional[str]
    attempts: int
    reflections: list = field(default_factory=list)

def run_reflexion(
    actor: Callable[[list], str],
    evaluator: Callable[[str], Optional[str]],
    reflector: Callable[[str, str], str],
    max_attempts: int = 4,
) -> ReflexionResult:
    """
    actor(reflections) -> output
    evaluator(output) -> None if pass, else a failure description
    reflector(output, failure) -> a specific reflection note
    """
    reflections = []

    for attempt in range(1, max_attempts + 1):
        output = actor(reflections)
        failure = evaluator(output)

        if failure is None:
            return ReflexionResult(True, output, attempt, reflections)

        note = reflector(output, failure)
        reflections.append(note)

    # Attempt limit reached; return the last attempt as the best effort.
    return ReflexionResult(False, output, max_attempts, reflections)

```

Applied to the sum-list example, `actor`, `evaluator`, and `reflector` would be swapped for real LLM calls, but the control flow — attempt, evaluate, reflect, retry, cap — stays identical.

Code Walkthrough

- `ReflexionResult` is just a small record of what happened: did it succeed, what was the final output, how many attempts did it take, and what did the agent learn along the way. Useful for logging and debugging your agent later.

- `actor(reflections)` takes the accumulated notes and produces a new attempt — this is where the reflection notes actually change behavior, by being part of the prompt/context passed in.
- `evaluator(output)` returns `None` on success or a description of the failure. This is deliberately generic — it can be a Python test runner, a rubric checker, or another LLM call acting as a judge.
- `reflector(output, failure)` is the "tutor writing in the margin" step — it turns a raw failure into a specific, reusable lesson.
- The `for` loop is the entire self-correction cycle: try, check, learn, repeat — bounded by `max_attempts` so it can never run away.
- Reflections accumulate in a list rather than replacing each other, so by attempt three the actor has the lessons from attempts one *and* two, not just the most recent one.

Dry Run

Using the sum-list example with `max_attempts=2`:

Attempt 1: - `reflections = []` - `actor([])` → returns the version that reads `nums[0]` directly
 - `evaluator(output)` → runs against `[]`, `[5]`, `[1,2,3]` → fails on `[]` with `IndexError` → returns "IndexError on empty list input" - `reflector(output, failure)` → "Function indexes `nums[0]` before checking length; must initialize `total=0` and handle empty input." - `reflections = ["Function indexes nums[0]..."]`

Attempt 2: - `actor(reflections)` → now sees the note in its prompt, writes the `total = 0` version - `evaluator(output)` → passes all three cases → returns `None` - Loop returns `ReflexionResult(success=True, output=<final code>, attempts=2, reflections=[...])`

Two attempts, one specific note, done.

Complexity Analysis

- **Time:** $O(k)$ actor calls and $O(k)$ evaluator calls, where k is the number of attempts actually used (bounded above by `max_attempts`). This is correct because each iteration performs exactly one actor call, one evaluator call, and — on failure — one reflector call, and the loop terminates the instant the evaluator passes or the cap is hit.
- **Space:** $O(k)$ to store the accumulated reflections, since each failed attempt appends one note. In practice k is small (3–5), so this is negligible — the real cost is the token budget consumed by re-

sending accumulated reflections in each new prompt, which grows linearly with the number of stored notes.

Alternative Solutions

Plain retry (covered above) is worth using when failures are genuinely stochastic — an unlucky sample, a formatting hiccup that resolves itself on a reroll. It's cheap and requires no extra infrastructure.

Fine-tuning solves a different problem entirely: it permanently updates the model's weights using a large set of examples, which costs time, data, and compute, but the improvement persists across every future task without needing a prompt-time note. Reflexion changes nothing about the model — it's a smarter prompt, not a smarter model — so the improvement only exists within the conversation or session where the reflection was written (unless you persist it to long-term memory).

The practical rule of thumb: if you need better results *this week*, reach for Reflexion. If you're seeing the same class of mistake across thousands of unrelated tasks and have the data and time to invest, fine-tuning is the better long-term fix.

Edge Cases

- **Empty input** — exactly the bug in our worked example; the evaluator must include an empty-input test case or this slips through silently.
- **Duplicate failures** — if the agent makes the same mistake twice in a row despite a reflection note, that's a signal the note wasn't specific enough, not that the loop needs more attempts.
- **Evaluator that never passes** — if the task is genuinely unsolvable or the evaluator is miscalibrated, the loop will exhaust `max_attempts` every time; make sure you're returning the best attempt rather than nothing.
- **Reflection note grows unbounded** — across many attempts, accumulated notes can bloat the prompt; consider summarizing or capping stored reflections for long loops.
- **Zero attempts allowed** — `max_attempts=0` should be rejected or treated as "run the actor once with no evaluation," not silently return an empty result.

Common Mistakes

1. **No stopping condition.** Letting the loop run unbounded burns tokens re-refining an answer that was already good enough.

2. **Vague reflections.** "The attempt was wrong" gives the next attempt nothing actionable — it has to name the specific failure.
3. **Treating Reflexion as a substitute for fine-tuning.** They solve different problems; reaching for the wrong one wastes time or money.
4. **Ignoring stochastic failures.** If a failure was pure bad luck, Reflexion's overhead (extra tokens, extra latency) isn't buying you anything over a plain retry.
5. **Discarding reflections between unrelated tasks.** If the same category of mistake keeps recurring across sessions, that's a sign you need long-term memory, not just short-term memory per task.

Interview Questions

- How would you design the evaluator for a task where "correctness" isn't a simple pass/fail test, like open-ended writing?
- What happens to this pattern if the evaluator itself is unreliable — how do you evaluate the evaluator?
- How would you decide when to promote a reflection from short-term to long-term memory?
- How does Reflexion's cost (tokens, latency) compare to a single well-engineered prompt, and when is the tradeoff worth it?
- How would you prevent reflection notes from growing unbounded across many attempts or many tasks?

Similar Problems

- **Self-consistency decoding** — sampling multiple outputs and taking a majority vote; related because it's another way of squeezing a better answer out of multiple attempts, but without any explicit feedback loop.
- **Tree-of-thought reasoning** — explores multiple reasoning paths rather than multiple full attempts; related in spirit (structured exploration beats one-shot generation) but different in mechanism.
- **Test-driven development (the human version)** — write a test, watch it fail, fix the code; Reflexion is essentially this loop automated and narrated in natural language by the agent itself.
- **Debugging with a linter/type checker** — an evaluator that catches specific, nameable failures before runtime, which is exactly the kind of specific signal a good reflection step needs.

Key Takeaways

Reflexion isn't a new model or a training technique — it's a loop: an actor attempts, an evaluator scores, and a reflection step writes down exactly what went wrong so the next attempt starts informed instead of blind. The two things that make or break it are a hard cap on attempts and reflections specific enough to actually change behavior. Used on the right kind of failure — systematic logic bugs rather than random noise — it turns a shaky agent into one that visibly gets better within a single session, at the cost of extra tokens and latency, and without touching a single training weight.

Watch the Video

If you want to see this pattern explained end-to-end with the full worked example, watch the video here: {LINK}

About the Series

This lesson is part of the Daily Python LeetCode series, where each entry breaks down one interview-relevant concept or problem — sometimes a classic algorithm, sometimes a systems idea like this one — with the intuition explained before a single line of code.

Call To Action

If this helped the Reflexion pattern click, like the video on YouTube, subscribe for the next lesson, and subscribe to the Substack so you don't miss future breakdowns. Drop a comment with the trickiest bug your own agent has confidently gotten wrong, and share this with a teammate who's building agents right now.

Quick Review

Pattern name: agent checks its own output and retries. Trigger phrase?

Reflexion Pattern — trigger: 'agent produces wrong answer with no self-check step'

Time complexity of one Reflexion loop with k retries?

$O(k)$ LLM calls — linear in max retries, each retry re-runs generate+critique+fix

Space complexity of Reflexion memory across attempts?

$O(k)$ — stores one reflection note per failed attempt until stop condition hits

Trickiest bug when implementing Reflexion loops?

No stopping point — loop retries forever; must cap attempts or detect no-progress convergence

Common mistake with the self-reflection note content?

Vague critique like 'try again' — note must name the specific error, not just flag failure

Reflexion vs plain retry-on-error: key difference?

Plain retry repeats same input; Reflexion injects a written critique into memory before next attempt

The Reflexion Pattern: How to Build a Self-Correcting AI Agent — free Python cheat sheet